



케이녹 포너블 1회차 과제

17기 최동현 (202511688)

1-1. Pwntools(pwn + tools, exploit tools) 설치 및 실행 환경

• Pwntools란?

: Python으로 로컬 프로세스, TCP 연결, ELF 분석, Packing, 어셈블, GDB 연동을 같은 코드 안에서 처리하는 **Exploit** 개발 라이브러리입니다. 공식 문서 기준 최신 안정판은 Python 3 사용을 권장하며 Ubuntu x86-64 환경에서 가장 많이 검증됩니다.

```
root@desktop-cyber: ~/workbench

root@desktop-cyber:~/workbench# python3 -m venv .venv
root@desktop-cyber:~/workbench# source .venv/bin/activate
(.venv) root@desktop-cyber:~/workbench# pip3 install --upgrade pip
Requirement already satisfied: pip in ./venv/lib/python3.10/site-packages (22.0.2)
Collecting pip
  Downloading pip-26.2.1-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB 15.3 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 22.0.2
    Uninstalling pip-22.0.2:
      Successfully uninstalled pip-22.0.2
Successfully installed pip-26.2.1
(.venv) root@desktop-cyber:~/workbench# pip3 install pwntools
Collecting pwntools
  Downloading pwntools-4.15.0-py2.py3-none-an
```

```
(.venv) root@desktop-cyber:~/workbench# python3
Python 3.10.12 (main, Jun 22 2026, 18:55:27) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from pwn import *
>>> print(pwnlib.__version__)
4.15.0
>>> 
```

1-2. Pwntools tube (process, remote, ssh)

• Pwntools tube 객체란?

: Pwntools의 tube 객체는 로컬 프로세스나 원격 서버와 데이터를 주고받기 위해 만든 연결 통로입니다. **process()**, **remote()**, **ssh.process()**처럼 입출력 통로가 다른 객체에 동일한 send/recv API를 제공합니다.

1) process([elf.path]) : 로컬 바이너리를 자식 프로세스로 실행

2) remote(host, port, timeout=3) : TCP 서버에 연결

3) ssh(user, host, port=22, password="pw") : SSH 서버에 로그인하여 원격 시스템에 접속

[ssh(...).process("/home/user/elf")]

```
1 #include <stdio.h>
2
3 int main(void) {
4     char name[16], ment[16];
5
6     printf("Name: ");
7     fflush(stdout); // flush output buffer
8     if(fgets(name, sizeof(name), stdin) == NULL) return 1;
9
10    printf("Ment: ");
11    fflush(stdout);
12    scanf("%s", ment);
13
14    printf("%s, %s", ment, name);
15    return 0;
16 }
```

```
1 from pwn import *
2
3 context.log_level = 'debug' # print debug log
4 p = process("./hello")
5
6 payload = b"K.knock"
7 log.info(payload)
8 log.info("K.knock".encode())
9 log.info(bytes("K.knock", "utf-8"))
10
11 p.recvuntil(b"Name: ")
12 p.sendline(payload)
13 p.recvuntil(b"Ment: ")
14
15 p.interactive()
```

```
(.venv) root@desktop-cyber:~/workbench# python3 tube_process.py
[+] Starting local process './hello' argv=[b'./hello'] : pid 149
/root/workbench/.venv/lib/python3.10/site-packages/pwnlib/log.py:
s
    self.log(logging.INFO, message, args, kwargs, 'info')
[*] K.knock
[*] K.knock
[*] K.knock
[DEBUG] Received 0x6 bytes:
    b'Name: '
[DEBUG] Sent 0x8 bytes:
    b'K.knock\n'
[DEBUG] Received 0x6 bytes:
    b'Ment: '
[*] Switching to interactive mode
$ Hello
[DEBUG] Sent 0x6 bytes:
    b'Hello\n'
[DEBUG] Received 0xf bytes:
    b'Hello, K.knock\n'
Hello, K.knock
[*] Process './hello' stopped with exit code 0 (pid 149225)
[*] Got EOF while reading in interactive
$
```

1-2. Pwntools tube (remote, ssh)

- Pwntools tube 객체 (remote, ssh)

- Remote

```
1 from pwn import *
2
3 context.log_level = 'debug'
4
5 HOST, PORT = "192.168.18.162", 8080
6 r = remote(HOST, PORT)
7
8 log.info(r.recvall())
9 r.interactive()
```

```
(.venv) root@desktop-cyber:~/workbench# python3 tube_remote.py
[+] Opening connection to 192.168.18.162 on port 8080: Done
[+] Receiving all data: Done (12B)
[DEBUG] Received 0xc bytes:
b'hello world!'
[*] Closed connection to 192.168.18.162 port 8080
/root/workbench/.venv/lib/python3.10/site-packages/pwnlib/log.py
s
self._log(logging.INFO, message, args, kwargs, 'info')
[*] hello world!
[*] Switching to interactive mode
[*] Got EOF while reading in interactive
$
```

- SSH

```
1 from pwn import *
2
3 #context.log_level = 'debug'
4
5 s = ssh(user='root', host='192.168.18.162', port=22, password='kali', raw=True)
6 #p = s.process("/root/PWN/hello")
7
8 sh = s.shell()
9 sh.interactive()
10 #p.interactive()
```

```
(.venv) root@desktop-cyber:~/workbench# python3 tube_ssh.py NOTERM
[x] Connecting to 192.168.18.162 on port 22
[+] Connecting to 192.168.18.162 on port 22: Done
[x] Opening new channel: 'shell'
[+] Opening new channel: 'shell': Done
[*] Switching to interactive mode
Linux attacker 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19
Last login: Mon Aug 17 16:23:50 2026 from 192.168.18.160

~(root@attacker)-[~]
#
```

1-3. Pwntools 송수신과 입출력 동기화

• 자주 사용하는 Tube 명령어

`recv(n, timeout)` : 현재 버퍼에서 최대 n바이트 수신

`recvline(keepends=True)` : 줄바꿈까지 수신 (keepends=False 는 newline 제거 Hello World**Wn**)

`recvuntil(delim, drop=False)` : 지정한 구분자를 만날 때까지 수신 (drop=True 는 구분자 제거)

`send(data) / sendline(data)` : 원본 바이트 / 바이트 뒤에 newline(**Wn**) 추가

`sendlineafter(delim, data)` : `recvuntil()` + `sendline()` 조합

```
1 from pwn import *
2
3 context.log_level = 'debug'
4 # print debug log (critical, debug, warn, info, notset[all] )
5 p = process("./hello")
6
7 p.sendlineafter(b"Name: ", b"K.knock")
8 p.sendlineafter(b"Ment: ", b"Welcome")
9
10 p.interactive()
```

```
(.venv) root@desktop-cyber:~/workbench# python3 tube_process.py
[+] Starting local process './hello' argv=[b'./hello'] : pid 149798
[DEBUG] Received 0x6 bytes:
      b'Name: '
[DEBUG] Sent 0x8 bytes:
      b'K.knock\n'
[DEBUG] Received 0x6 bytes:
      b'Ment: '
[DEBUG] Sent 0x8 bytes:
      b'Welcome\n'
[*] Switching to interactive mode
[*] Process './hello' stopped with exit code 0 (pid 149798)
[DEBUG] Received 0x11 bytes:
      b'Welcome, K.knock\n'
Welcome, K.knock
[*] Got EOF while reading in interactive
$
```

멈춤 사항

수신 구분자와 제한 시간(timeout)을 명시하면 출력 지연이나 네트워크 분할 때문에 풀이가 멈추는 문제를 줄일 수 있습니다. 때문에 pwntools 기본 송수신 값은 bytes이므로 인코딩 경계를 명확하게 관리해야 합니다.

1-4. context와 ELF 분석 기능

• context 설정과 ELF() 객체

: context는 아키텍처, 운영체제, 비트 수, 엔디안을 pwntools 전체에 적용합니다. ELF()는 실행 파일의 헤더·심볼·GOT·PLT·섹션을 읽어 주소 계산과 보호기법 확인에 사용합니다.

```
1 from pwn import *
2
3 context.log_level = 'debug'
4
5 e = ELF("./hello", checksec=False)
6 text = e.get_section_by_name(".text")
7
8 log.info(f"base addr: {hex(e.address)}")
9 log.info(f"entrypoint addr: {hex(e.entry)}")
10 log.info(f".text addr: {hex(text.header.sh_addr)}")
11 log.info(e.functions["main"])
12 log.info(f"main addr: {hex(e.sym.main)}") # main addr
13 log.info(f"printf plt addr: {hex(e.plt.printf)}") # plt stub (code addr)
14 log.info(f"printf got addr: {hex(e.got.printf)}") # got entry (func addr)
```

```
(.venv) root@desktop-cyber:~/workbench# python3 elf_info.py
[*] base addr: 0x400000
[*] entrypoint addr: 0x401070
[*] .text addr: 0x401070
[*] Function(name='main', address=0x401156, size=0xb3, elf=ELF('/root/workbench/hello'))
[*] main addr: 0x401156 base + 0x1156
[*] printf plt addr: 0x401030
[*] printf got addr: 0x404018
```

* PIE 바이너리

ELF의 심볼 값은 파일 기준 오프셋일 수 있습니다. 주소 누출로 베이스를 구한 뒤 `elf.address = base`를 설정하면 `elf.sym`, `elf.got` 값이 재계산됩니다.

PIE/ASLR

하드코딩한 절대 주소는 PIE/ASLR 환경에서 재실행 시 달라집니다. 때문에 ELF 메타데이터, 실제 누출 주소, 해당 실행의 라이브러리 버전을 함께 확인해야 합니다.

1-5. packing(), flat(), cyclic()

• 주소와 패턴을 바이트열로 만드는 기능 (Packing)

: p64()/u64()는 정수와 8바이트 값을 변환하고 flat()은 여러 값과 패딩을 하나의 payload로 결합합니다. cyclic()은 충돌 위치를 찾기 위한 비반복 패턴을 생성합니다.

```
1 from pwn import *
2
3 context.arch = "amd64"
4 context.log_level = 'debug'
5
6 addr = 0x401000
7 packed = p64(addr)
8
9 log.info(packed)
10 log.info(hex(u64(packed)))
11
12 log.info(flat({40: [packed]})) # offset 40 + packed
13 log.info(cyclic(60, n=4))
14 log.info(cyclic_find(b"kaaa", n=4))
```

* 주의 사항!

생성과 탐색에서 n 값을 동일하게 사용합니다. 누출 바이트를 u64에 전달할 때는 8바이트로 패딩하고, 엔디안과 비트 수를 context에 먼저 설정합니다.

```
(.venv) root@desktop-cyber:~/workbench# python3 packing.py
/root/workbench/.venv/lib/python3.10/site-packages/pwnlib/log.py:396: Bytes
s
self._log(logging.INFO, message, args, kwargs, 'info')
[*] \x00\x10@\x00\x00\x00\x00\x00
[*] 0x401000
[*] aaaabaaacaaadaaaeaaafaaagaaahaaiaaaiaaaa\x00\x10@\x00\x00\x00\x00\x00
[*] aaaabaaacaaadaaaeaaafaaagaaahaaiaaaiaaaakaaalaaaamaanaaaaoaaa
[*] 40
```

> Payload 순서

1. cyclic(offset, n)을 입력
2. corefile 또는 GDB에서 손상된 값 확인
3. cyclic_find(value, n) 실행
4. flat({offset: [...]})으로 payload 구성

충돌 위치

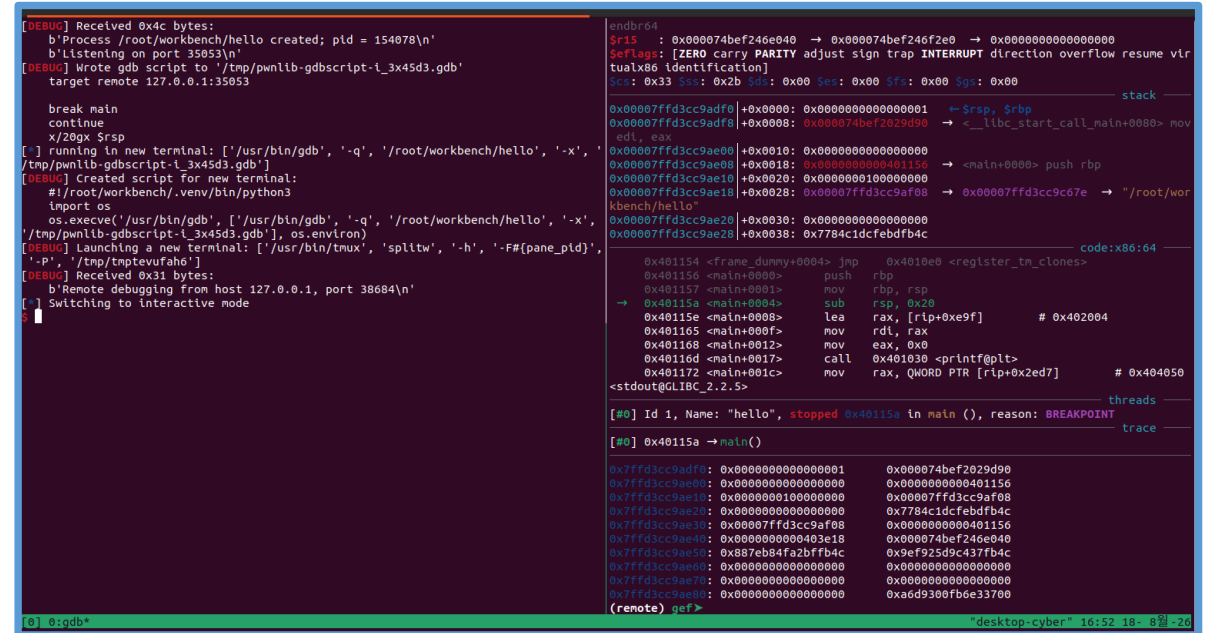
cyclic은 허가된 로컬 프로세스의 충돌 위치 확인에만 사용됩니다. 외부 서비스에 큰 패턴을 반복 전송하면 장애를 유발할 수 있으므로 입력 크기와 횟수를 제한해야 합니다.

1-6. pwntools GDB 연동 (debug, attach)

- gdb.debug()와 gdb.attach()

: gdb.debug()는 gdbserver로 프로그램을 처음부터 디버깅하고 gdb.attach()는 이미 실행 중인 로컬 프로세스에 연결합니다. gdbscript에 중단점·표시 방식·continue 명령을 기록하면 동일한 분석 상태를 반복할 수 있습니다.

```
1 from pwn import *
2
3 context.binary = e = ELF("./hello", checksec=False)
4 context.terminal = ["tmux", "splitw", "-h"]
5 context.log_level = 'debug'
6
7 gdbscript = """
8 break main
9 continue
10 x/20gx $rsp
11 """
12
13 g = gdb.debug([e.path], gdbscript=gdbscript)
14 # [gdb.attach]
15 # p = process("./hello")
16 # g = gdb.attach(p, gdnscrip=gdbscript)
17 g.interactive()
```



```
[DEBUG] Received 0x4c bytes:
b'Process /root/workbench/hello created; pid = 154078\n'
b'Listening on port 35053\n'
[DEBUG] Wrote gdb script to '/tmp/pwnlib-gdbscript-t_3x45d3.gdb'
target remote 127.0.0.1:35053

break main
continue
x/20gx $rsp
[.] running in new terminal: ['/usr/bin/gdb', '-q', '/root/workbench/hello', '-x',
/tmp/pwnlib-gdbscript-t_3x45d3.gdb']
[DEBUG] Created script for new terminal:
# /root/workbench/.venv/bin/python3
import os
os.execcve(['/usr/bin/gdb', ['/usr/bin/gdb', '-q', '/root/workbench/hello', '-x',
'/tmp/pwnlib-gdbscript-t_3x45d3.gdb'], os.environ)
[DEBUG] Launching a new terminal: ['/usr/bin/tmux', 'splitw', '-h', '-F#{pane_pid}',
'-P', '/tmp/tmp7evufah0']
[DEBUG] Received 0x31 bytes:
b'Remote debugging from host 127.0.0.1, port 38684\n'
[.] Switching to interactive mode

endbrq4
$ri15 : 0x000074bef246e040 -> 0x000074bef246f2e0 -> 0x0000000000000000
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume vir
tualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00
stack
0x00007ffdf3cc9adfe -> 0x0000000000000001 -> $rsp, $rbp
0x00007ffdf3cc9adfe -> 0x000074bef2429d90 -> <__libc_start_call_main+0080> mov
edi, eax
0x00007ffdf3cc9ae00 -> 0x0000000000000000
0x00007ffdf3cc9ae08 -> 0x000000000000401156 -> <main+0000> push rbp
0x00007ffdf3cc9ae10 -> 0x0000000010000000
0x00007ffdf3cc9ae18 -> 0x00007ffdf3cc9af08 -> 0x00007ffdf3cc9c67e -> "/root/wor
kbench/hello"
0x00007ffdf3cc9ae20 -> 0x0000000000000000
0x00007ffdf3cc9ae28 -> 0x00007ffdf3cc9ae28 -> 0x7784c1dcfebf4c
code:x86:64
0x401154 <frame_dummy+0004> jmp 0x4010e0 <register_tm_clones>
0x401156 <main+0000> push rbp
0x401157 <main+0001> mov rbp, rsp
0x401159 <main+0004> sub rsp, 0x20
-> 0x40115a <main+0008> lea rax, [rip+0xe9f] # 0x402004
0x401165 <main+000f> mov rdi, rax
0x401168 <main+0012> mov eax, 0x0
0x40116d <main+0017> call 0x401030 <printf@plt>
0x401172 <main+001c> mov rax, QWORD PTR [rip+0x2ed7] # 0x404050
<stdout@GLIBC_2.2.5>
threads
[#0] Id 1, Name: "hello", stopped 0x40115a in main (), reason: BREAKPOINT
trace
[#0] 0x40115a -> main()
0x7ffdf3cc9adfe: 0x0000000000000001 0x000074bef2429d90
0x7ffdf3cc9ae00: 0x0000000000000000 0x000000000000401156
0x7ffdf3cc9ae10: 0x0000000010000000 0x00007ffdf3cc9af08
0x7ffdf3cc9ae20: 0x0000000000000000 0x7784c1dcfebf4c
0x7ffdf3cc9ae30: 0x00007ffdf3cc9af08 0x000000000000401156
0x7ffdf3cc9ae40: 0x000000000000403e18 0x000074bef246e040
0x7ffdf3cc9ae50: 0x887eb84fa2bffb4c 0x9ef925d9c437fb4c
0x7ffdf3cc9ae60: 0x0000000000000000 0x0000000000000000
0x7ffdf3cc9ae70: 0x0000000000000000 0x0000000000000000
0x7ffdf3cc9ae80: 0x0000000000000000 0xa6d930fb6e33700
(remote) gef>
```

* attach 범위

일반적인 인터넷 원격 CTF 환경에는 직접 attach할 수 없습니다. 대상과 GDB가 같은 시스템에 있거나, 별도의 gdbserver/SSH 디버깅 권한이 있어야 합니다.

GDBScript 디버깅 중 ASLR을 끈 상태와 실제 실행 환경을 구분합니다. gdbscript의 주소가 PIE 베이스를 전제로 하는지 확인해야 합니다.

2-1. read 시스템 콜 동작

- read()란?

: 파일 디스크립터 fd에서 최대 count바이트를 buf 주소로 복사하고, 성공하면 실제 읽은 바이트 수를 반환합니다. 반환값이 count보다 작아도 오류가 아니며, 0은 EOF, -1은 Error를 의미합니다.

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void) {
5     char buf[16];
6     printf("input: ");
7     read(STDIN_FILENO, buf, 64);
8     printf("%s", buf);
9     return 0;
10 }
```

```
(.venv) root@desktop-cyber:~/workbench# echo "test" | ./read_func
input: test
```

2-3. scanf 계열 함수의 입력 규칙

• scanf()란?

: 표준 입력의 문자를 format에 따라 변환해 포인터가 가리키는 객체에 저장하고, 성공적으로 대입한 항목 수를 반환합니다. 입력 전체 길이나 소비한 문자 수를 반환하지 않으므로 반환값과 남은 입력을 별도로 관리해야 합니다.

```
1 #include <stdio.h>
2
3 int main(void) {
4     char safe[8];
5     char unsafe[8];
6     int num;
7
8     if (scanf("%s", unsafe) != 1) return 1;
9     if (scanf("%7s", unsafe) != 1) return 1;
10
11     if (scanf("%d", &num) != 1) {
12         fputs("\n", stderr);
13         return 1;
14     }
15
16     return 0;
17 }
```

%s 규칙

- 앞의 공백을 건너뛰
- 다음 공백 전까지 저장
- field width가 없으면 배열 크기를 모름
- char[8]에는 %7s 사용 : 문자 7개 + NULL 문자 1개

%d 규칙

- 변환 실패 시 0을 반환할 수 있음
- 대상 인자의 주소(&number)를 전달

3-1. ELF 보호기법과 checksec

• checksec로 확인하는 항목

: checksec는 실행 파일의 ELF 속성을 검사해 RELRO, Stack Canary, NX, PIE, FORTIFY 등을 요약합니다. 취약점 존재 여부를 판정하는 도구가 아니라 컴파일·링크 보안 설정을 확인하는 도구입니다.

NX	스택 등 데이터 영역의 실행 권한 제한
Canary	함수 반환 전 스택 손상 검사
PIE + ASLR	코드·스택·라이브러리 주소 무작위화
RELRO	동적 재배치 영역의 쓰기 가능 범위 제한
FORTIFY	일부 libc 호출에 객체 크기 검사 추가

* 해석 순서

보호기법 상태 확인 → 입력 경계·메모리 버그 확인 → 필요한 정보 누출·제어 가능성 검증. checksec 결과만으로 exploit 가능 여부를 단정하지 않습니다.

```
(.venv) root@desktop-cyber:~/workbench# checksec ./hello
[*] '/root/workbench/hello'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:  No

(.venv) root@desktop-cyber:~/workbench# python3
Python 3.10.12 (main, Jun 22 2026, 18:55:27) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from pwn import *
>>> print(ELF("./hello").checksec())
[*] '/root/workbench/hello'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:  No
```

보안 관점

모든 보호가 켜져 있어도 취약한 코드는 수정되지 않습니다. 보호기법은 공격에 필요한 조건을 늘리므로, 원인은 경계 검사와 메모리 안전성 수준에서 제거해야 합니다.

3-2. NX (Non-Executable Memory)

• NX란?

: 메모리 페이지 권한에서 쓰기(W)와 실행(X)을 분리해 스택·힙에 주입한 바이트를 CPU가 바로 명령어로 실행하지 못하게 합니다. ELF의 GNU_STACK 프로그램 헤더 권한으로 스택 실행 요구 여부를 확인할 수 있습니다.

```
(.venv) root@desktop-cyber:~/workbench# readelf -W -l ./hello | grep GNU_STACK
GNU_STACK      0x00000000 0x0000000000000000 0x0000000000000000 0x00000000 0x00000000 RWE 0x10
(.venv) root@desktop-cyber:~/workbench# checksec ./hello
[*] '/root/workbench/hello'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:  No
(.venv) root@desktop-cyber:~/workbench#
```

실행 흐름

1. 입력 데이터는 스택에 저장될 수 있음
2. 반환 주소가 스택 주소로 변경됨
3. 해당 페이지에 X 권한이 없으면 실행 단계에서 예외 발생

NX가 막지 않는 것

- 기존 실행 코드 재사용(ROP)
- 메모리 읽기·쓰기 자체

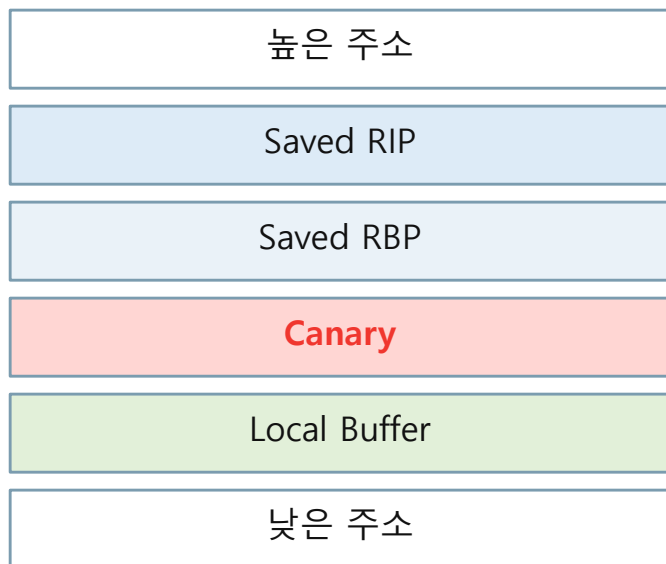
보안 관점

execstack는 셸코드 실습처럼 목적이 분명한 격리 환경에서만 사용합니다. 배포 바이너리는 noexecstack을 유지하고 JIT처럼 실행 메모리가 필요한 경우에도 WX 정책을 적용합니다.

3-3. Stack Canary

- Canary란?

: 보호 대상 함수 진입 시 스택 프레임에 비밀 값을 저장하고, 반환 직전에 원본과 비교합니다. 버퍼 오버플로가 Canary까지 덮으면 `__stack_chk_fail()` 이 호출되어 정상 반환 전에 프로세스가 종료됩니다.



보안 관점

Canary 값이 노출되면 같은 실행에서 재사용될 수 있고, Canary 전에 위치한 변수는 손상될 수 있습니다. 근본 대응은 입력 길이 검증이며 Canary는 추가 방어 계층입니다.

3-4. PIE와 ASLR

• PIE / ASLR이란?

: PIE(Position Independent Executable)는 실행 파일 코드를 임의 베이스에 배치할 수 있게 만들고, ASLR은 실행할 때 스택·mmap·공유 라이브러리·PIE 코드·힙 등의 주소를 무작위화합니다. PIE 빌드와 커널 ASLR 설정이 함께 있어야 코드 베이스가 바뀝니다.

```
(.venv) root@desktop-cyber:~/workbench# gcc -pie ./hello.c -o ./hello_pie
(.venv) root@desktop-cyber:~/workbench# gcns ./hello.c -o ./hello
(.venv) root@desktop-cyber:~/workbench# readelf -h ./hello_pie | grep Type
Type:                                DYN (Position-Independent Executable file)
(.venv) root@desktop-cyber:~/workbench# readelf -h ./hello | grep Type
Type:                                EXEC (Executable file)
(.venv) root@desktop-cyber:~/workbench#
```

보안 관점

주소 무작위화는 정보 유출이 없을 때 효과가 큼니다. 포맷 스트링, 포인터 출력, 오류 로그 등 주소 누출 원인을 제거하고 PIE를 배포 기본값으로 사용합니다.

3-5. RELRO (Relocation Read-Only)

• RELRO란?

: ELF 동적 링커가 재배포 작업에 사용하는 영역을 초기화 후 읽기 전용으로 전환합니다. Partial RELRO는 일부 영역만 보호하고 lazy binding이 남을 수 있으며, Full RELRO는 시작 시 재배포를 완료하고 GOT를 읽기 전용으로 만듭니다.

```
(.venv) root@desktop-cyber:~/workbench# readelf -W -l ./hello | grep RELRO
GNU_RELRO      0x002e10 0x00000000000403e10 0x00000000000403e10 0x0001f0 0x0001f0 R    0x1
(.venv) root@desktop-cyber:~/workbench#
```

보호 효과

- GOT 엔트리 덮어쓰기 경로 제한
- 동적 심볼을 프로그램 시작 시 해결

제한

- 다른 쓰기 가능한 데이터 손상은 별도 문제
- 취약한 메모리 쓰기 원인을 제거하지 않음

보안 관점

배포 빌드는 -Wl,-z,relro,-z,now를 사용해 Full RELRO를 적용합니다. 시작 시간과의 작은 트레이드오프보다 GOT 무결성 확보가 일반적인 보안 기본값에 가깝습니다.

4-1. Linux x86-64 syscall

- syscall이란?

: 사용자 모드 프로그램이 커널 기능을 요청하는 인터페이스입니다. x86-64 Linux에서는 syscall 명령으로 전환하며, 시스템 콜 번호는 rax, 인자는 rdi·rsi·rdx·r10·r8·r9, 반환값은 rax를 사용합니다.

rax	syscall 번호 / 반환값	rdi	1번째 인자	rsi	2번째 인자
rdx	3번째 인자	r10	4번째 인자	r8 / r9	5·6번째 인자

보안 관점

시스템 콜 허용 범위는 seccomp 같은 정책으로 줄일 수 있습니다. syscall 번호와 레지스터 규약은 아키텍처마다 다르므로 x86-64 값을 다른 환경에 그대로 적용하지 않습니다.

4-2. shellcode와 pwntools shellcraft

• shellcode란?

: 메모리에 바이트열로 배치되어 직접 실행되는 짧은 기계어 코드입니다. 위치 독립성, 허용되지 않는 바이트, 레지스터 상태, 메모리 실행 권한, 시스템 콜 규약을 함께 고려해야 합니다.

```
1 from pwn import *
2
3 context.clear(arch="amd64", os="linux")
4 source = shellcraft.echo("SHELLCODE_OK\n")
5 source += shellcraft.exit(0)
6 code = asm(source)
7
8 print(f"length: {len(code)}")
9 print(disasm(code))
10 sh = run_shellcode(code)
11 print(sh.recvall().decode())
```

의미

- shellcraft: 아키텍처별 어셈블리 생성
- asm: 어셈블리 → bytes
- disasm: bytes → 명령어 확인
- run_shellcode: 로컬 임시 ELF에서 실행

```
(.venv) root@desktop-cyber:~/workbench# python3 shellcode.py
length: 57
0: 48 b8 01 01 01 01 01 01 01 01  movabs rax, 0x101010101010101
a: 50                                push  rax
b: 48 b8 44 5e 4e 4a 0b 01 01 01  movabs rax, 0x101010b4a4e5e44
15: 48 31 04 24                       xor   QWORD PTR [rsp], rax
19: 48 b8 53 48 45 4c 4c 43 4f 44  movabs rax, 0x444f434c4c454853
23: 50                                push  rax
24: 6a 01                            push  0x1
26: 58                                pop   rax
27: 6a 01                            push  0x1
29: 5f                                pop   rdi
2a: 6a 0d                            push  0xd
2c: 5a                                pop   rdx
2d: 48 89 e6                         mov   rsi, rsp
30: 0f 05                           syscall
32: 31 ff                            xor   edi, edi
34: 6a 3c                            push  0x3c
36: 58                                pop   rax
37: 0f 05                           syscall
[*] '/tmp/pwn-asm-k9xm5ihi/step3-elf'
Arch:      amd64-64-little
RELRO:     No RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
[+] Starting local process '/tmp/pwn-asm-k9xm5ihi/step3-elf': pid 154529
[+] Receiving all data: Done (13B)
[*] Process '/tmp/pwn-asm-k9xm5ihi/step3-elf' stopped with exit code 0 (pid 154529)
SHELLCODE_OK
```

보안 관점

실습 코드는 자신의 로컬 프로세스에서만 실행합니다. 실제 배포 환경은 NX, 최소 권한, seccomp, 입력 검증을 함께 적용하며 외부에서 받은 바이트열을 실행하지 않습니다.

5. GCC 디버그·검사·보안 옵션

• 옵션을 목적별로 구분

<code>-g3</code>	GDB용 디버그 정보와 매크로 정보	<code>-O0</code>	최적화를 최소화해 소스와 실행 흐름 대응
<code>-Wall -Wextra -Wpedantic</code>	경고 범위 확대	<code>-fsanitize=address,undefined</code>	메모리 오류·정의되지 않은 동작 검사
<code>-fno-omit-frame-pointer</code>	스택 추적 보존	<code>-fstack-protector-strong</code>	Canary 적용 범위 확대
<code>-fPIE -pie</code>	PIE 코드 생성과 PIE 링크	<code>-Wl,-z,relro,-z,now</code>	링커에 Full RELRO 옵션 전달

버그 찾기 빌드

```
$ gcc -g3 -O0 -Wall -Wextra -Wpedantic -fsanitize=address,undefined -fno-omit-frame-pointer -o app_dbg app.c
```

배포 보안 빌드

```
$ gcc -O2 -D_FORTIFY_SOURCE=3 -fstack-protector-strong -fPIE -pie -Wl,-z,relro,-z,now -Wl,-z,noexecstack -o app app.c  
$ checksec --file=./app
```

보안 관점

Sanitizer는 테스트 빌드에서 버그를 찾는 도구이고, hardening 옵션은 배포 공격 난도를 높이는 설정입니다. 둘 중 하나만 적용하지 않고 소스 수정·테스트·보안 링크를 함께 수행합니다.



감사합니다

17기 최동현